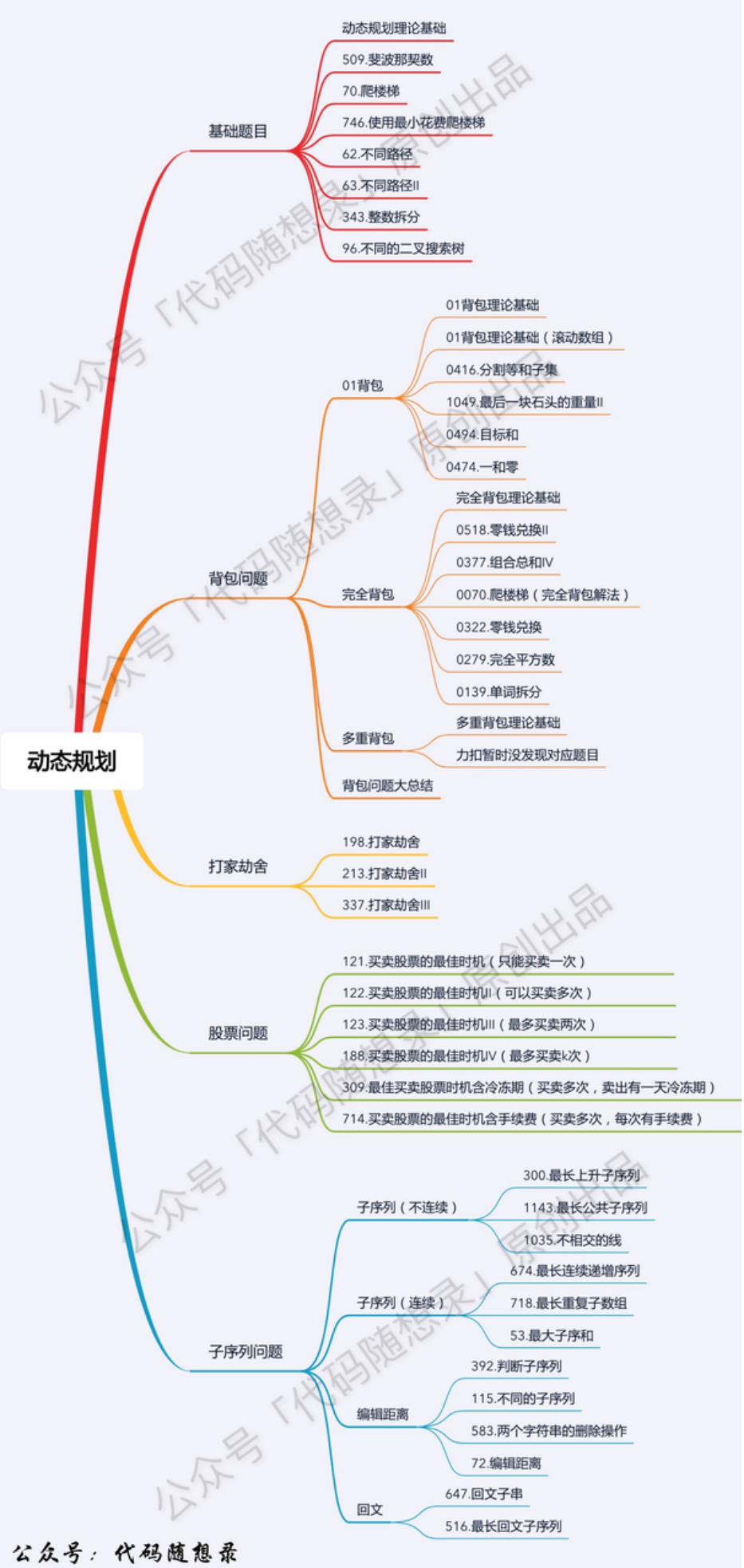
动态规划

经典的动态规划题，详细的解题思路和代码。声明：部分参考代码随想录里的题单分类！


动态规划
基础题目
动态规划理论基础
509.斐波那契数
70.爬楼梯
746.使用最小花费爬楼梯
62.不同路径
63.不同路径II
343.整数拆分
96.不同的二叉搜索树
背包问题
01背包
01背包理论基础
01背包理论基础（滚动数组）
0416.分割等和子集
1049.最后一块石头的重量II
0494.目标和
0474.一和零
完全背包
完全背包理论基础
0518.零钱兑换II
0377.组合总和IV
0070.爬楼梯（完全背包解法）
0322.零钱兑换
0279.完全平方数
0139.单词拆分
多重背包
多重背包理论基础
力扣暂时没发现对应题目
背包问题大总结
打家劫舍
198.打家劫舍
213.打家劫舍II
337.打家劫舍III
股票问题
121.买卖股票的最佳时机（只能买卖一次）
122.买卖股票的最佳时机II（可以买卖多次）
123.买卖股票的最佳时机III（最多买卖两次）
188.买卖股票的最佳时机IV（最多买卖k次）
309.最佳买卖股票时机含冷冻期（买卖多次，卖出有一天冷冻期）
714.买卖股票的最佳时机含手续费（买卖多次，每次有手续费）
子序列问题
子序列（不连续）
300.最长上升子序列
1143.最长公共子序列
1035.不相交的线
子序列（连续）
674.最长连续递增序列
718.最长重复子数组
53.最大子序和
编辑距离
392.判断子序列
115.不同的子序列
583.两个字符串的删除操作
72.编辑距离
回文
647.回文子串
516.最长回文子序列
公众号：代码随想录

DP = 记住子问题答案，避免重复计算！

把“全局最优”拆成“局部最优”来推。

1.求最值问题

2.能不能定义“以 i 结尾/开始”的状态?

3.当前结果是否依赖之前?

4.有没有重复的问题

基础题目：

509.斐波那契数列

斐波那契数 （通常用 `F(n)` 表示）形成的序列称为 **斐波那契数列** 。该数列由 `0` 和 `1` 开始，后面的每一项数字都是前面两项数字的和。也就是：

F(0) = 0，F(1) = 1

F(n) = F(n - 1) + F(n - 2)，其中 n > 1

给定 `n` ，请计算 `F(n)` 。

代码块

```
class Solution:
    def fib(self, n: int) -> int:
        f =[0]*(n+1)
        if n==0:
            return 0
        if n==1:
            return 1    #注意边界问题
        f[0] =0
        f[1] =1
        for i in range(2,n+1):
            f[i] = f[i-1]+f[i-2]
        return f[n]
```

70.爬楼梯

假设你正在爬楼梯。需要 `n` 阶你才能到达楼顶。

每次你可以爬 `1` 或 `2` 个台阶。你有多少种不同的方法可以爬到楼顶呢?

示例 1：

输入： n = 2
输出： 2

解释：有两种方法可以爬到楼顶。
1. 1 阶 + 1 阶
2. 2 阶

> 分类讨论：
>
> 如果最后一步爬了 1 个台阶，那么我们得先爬到 i－1，要解决的问题缩小成：从 0 爬到 i－1 有多少种不同的方法。
>
> 如果最后一步爬了 2 个台阶，那么我们得先爬到 i－2，要解决的问题缩小成：从 0 爬到 i－2 有多少种不同的方法。

代码块

```
def climbStairs(self, n: int) -> int:
        f=[0]*(n+1)
        f[0] =1  #第零阶台阶
        f[1] =1
        for i in range(2,n+1):
            f[i] = f[i-1]+f[i-2]
        return f[n]
```

爬楼梯II

题目描述

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬至多m (1 <= m < n)个台阶。你有多少种不同的方法可以爬到楼顶呢?

注意：给定 n 是一个正整数。

输入描述

输入共一行，包含两个正整数，分别表示n, m

输出描述

输出一个整数，表示爬到楼顶的方法数。

输入示例

代码块

```
3 2
```

输出示例

代码块

```
3
提示信息
数据范围:
1 <= m < n <= 32;
当 m = 2, n = 3 时, n = 3 这表示一共有三个台阶, m = 2 代表你每次可以爬一个台阶或者两个台阶。
此时你有三种方法可以爬到楼顶。
1 阶 + 1 阶 + 1 阶段
1 阶 + 2 阶
2 阶 + 1 阶
```

代码块

```python
import sys
n,m = map(int,sys.stdin.readline().split())
dp =[0]*(n+1)
dp[0] =1
dp[1] =1
for i in range(2,n+1):
    for j in range(1,m+1):
        if i>=j:
            dp[i] +=dp[i-j]
print(dp[n])
```

764.使用最小花费爬楼梯

给你一个整数数组 `cost` ，其中 `cost[i]` 是从楼梯第 `i` 个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个或者两个台阶。

你可以选择从下标为 `0` 或下标为 `1` 的台阶开始爬楼梯。

请你计算并返回达到楼梯顶部的最低花费。

示例 1：

输入： cost = [10,***15***,20]
输出： 15
解释： 你将从下标为 1 的台阶开始。
- 支付 15 ，向上爬两个台阶，到达楼梯顶部。
总花费为 15 。

代码块

```python
class Solution:
    #注意这道题到达顶层是n, 不是n-1,cost[]:表示0-n-1台阶往上跳需要的花费
```

```python
    #dfs(i):表示到达i楼层的最小花费
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache  # 缓存装饰器，避免重复计算 dfs 的结果
        def dfs(i:int)->int:
            if i<=1:
                return 0 #表示跳到0，1台阶的花费为0，因为一开始可以站在0，1台阶上往上跳
            return min(dfs(i-1)+cost[i-1],dfs(i-2)+cost[i-2])
        return dfs(len(cost))
```

代码块

```python
class Solution:
    #注意这道题到达顶层是n，不是n-1,cost[]:表示0-n-1台阶往上跳需要的花费
    #f[i]:表示到达i楼层的最小花费
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0]*(n+1)
        f[0] =0  #到达0阶台阶的花费
        f[1] =0 #到达1阶台阶的花费
        for i in range(2,n+1):
            f[i] = min(f[i-1]+cost[i-1],f[i-2]+cost[i-2])
        return f[n]
```

62.不同路径

一个机器人位于一个 `m x n` 网格的左上角 （起始点在下图中标记为 “Start” ）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish” ）。

问总共有多少条不同的路径?

示例 1：

> **输入：** m = 3, n = 7
> **输出：** 28

代码块

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        ####dp[i][j]:表示到达[i,j]位置的路径个数
        ####其中第一行和第一列中的元素只有一条路径到达，因为智能往下和往走
        #1.定义
        dp =[[0]*n for _ in range(m)]
        #2.初始化
        for i in range(m):
            dp[i][0] =1
        for j in range(n):
            dp[0][j] =1
        #3.递归公式
        for i in range(1,m):
            for j in range(1,n):
                dp[i][j] = dp[i-1][j]+dp[i][j-1]
        #4.返回值
        return dp[m-1][n-1]
```

63.不同路径II

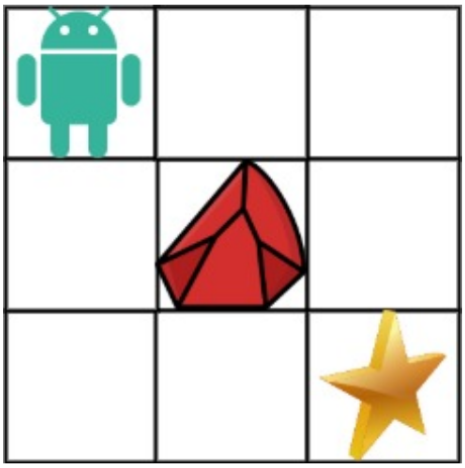
给定一个 `m x n` 的整数数组 `grid`。一个机器人初始位于 **左上角**（即 `grid[0][0]`）。机器人尝试移动到 **右下角**（即 `grid[m - 1][n - 1]`）。机器人每次只能向下或者向右移动一步。

网格中的障碍物和空位置分别用 `1` 和 `0` 来表示。机器人的移动路径中不能包含 **任何** 有障碍物的方格。

返回机器人能够到达右下角的不同路径数量。

测试用例保证答案小于等于 `2 * 10(9)` 。

示例 1：



> **输入：** obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
> **输出：** 2
> **解释：** 3x3 网格的正中间有一个障碍物。
> 从左上角到右下角一共有 `2` 条不同的路径：
> 1. 向右 -> 向右 -> 向下 -> 向下
> 2. 向下 -> 向下 -> 向右 -> 向右

代码块

```
class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        #1.定义
        m = len(obstacleGrid)
        n =len(obstacleGrid[0])
        dp =[[0]*n for _ in range(m)]
        #2.初始化
        for i in range(m):
            if obstacleGrid[i][0]==0:
                dp[i][0]=1
            else:
                break  #要及时跳出循环，因为有障碍物，后面的路就不能走了
        for j in range(n):
            if obstacleGrid[0][j]==0:
                dp[0][j]=1
            else:
                break #要及时跳出循环，因为有障碍物，后面的路就不能走了
        #3.递推公式
        for i in range(1,m):
            for j in range(1,n):
                if obstacleGrid[i][j]==0:
                    dp[i][j] = dp[i-1][j]+dp[i][j-1]
        #4.输出
        return dp[-1][-1]
```

343.整数拆分

给定一个正整数 `n` ，将其拆分为 `k` 个 **正整数** 的和（ `k >= 2` ），并使这些整数的乘积最大化。

返回 *你可以获得的最大乘积* 。

示例 1:

输入: n = 2
输出: 1
解释: 2 = 1 + 1, 1 × 1 = 1。

动态规划思路分析

对于的正整数 n，当 n≥2 时，可以拆分成至少两个正整数的和。令 k 是拆分出的第一个正整数，则剩下的部分是 n−k，n−k 可以不继续拆分，或者继续拆分成至少两个正整数的和（一个问题可以分解为相似的子问题因此想到动态规划）。由于每个正整数对应的最大乘积取决于比它小的正整数对应的最大乘积，因此可以使用动态规划求解。

算法实现步骤：

1.确定dp数组以及下标的含义

dp[i] 表示将正整数 i 拆分成至少两个正整数的和之后，这些正整数的最大乘积

2.确定状态转移方程

当 i≥2 时，假设对正整数 i 拆分出的第一个正整数是 j（1≤j<i），则有以下两种方案：

将 i 拆分成 j 和 i−j 的和，且 i−j 不再拆分成多个正整数，此时的乘积是 j×(i−j)；

将 i 拆分成 j 和 i−j 的和，且 i−j 继续拆分成多个正整数，此时的乘积是 j×dp[i−j]。

因此，当 j 固定时，有 dp[i]=max(j×(i−j),j×dp[i−j])。由于 j 的取值范围是 1 到 i−1，需要遍历所有的 j 得到 dp[i]的

3.初始化状态

0 不是正整数，1 是最小的正整数，0 和 1 都不能拆分，因此 dp[0]=dp[1]=0。

4.遍历顺序

由状态转移方程知道 dp[i] 是从 j×(i−j) 和 j×dp[i−j] 且 j 的取值范围是 1 到 i−1，需要遍历所有的 j 得到 dp[i] 所以从前往后遍历。

5.返回值

最终得到 dp[n] 的值即为将正整数 n 拆分成至少两个正整数的和之后，这些正整数的最大乘积。

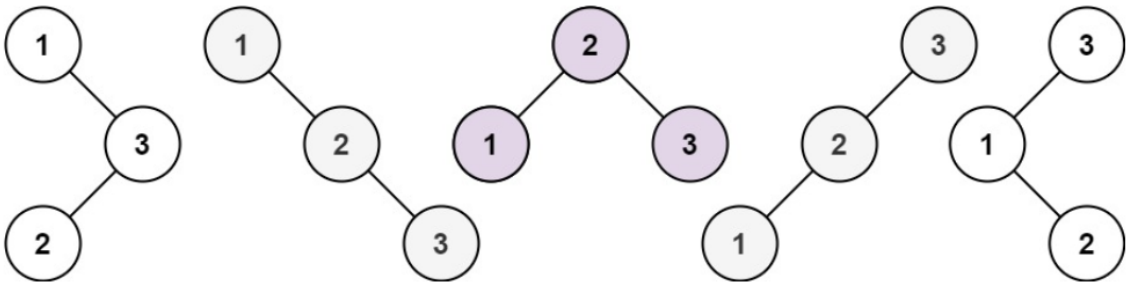
代码块

```python
class Solution:
    def integerBreak(self, n: int) -> int:
    #dp[i]:表示i拆分为2个或2个以上后，乘积的最大值
        dp =[0]*(n+1)
        dp[2] =1
        for i in range(3,n+1):
            for j in range(1,i):
                dp[i] = max(dp[i],j*(i-j),j*dp[i-j])
                #要和dp[i]比较，因为当你不断枚举到一个新的 j 时，必须拿“之前已经找到的最优值”跟“当前这种拆法的值”比较，取更大的那个。
        return dp[n]
```

96.不同的二叉搜索树

给你一个整数 `n` ，求恰由 `n` 个节点组成且节点值从 `1` 到 `n` 互不相同的 **二叉搜索树** 有多少种？返回满足题意的二叉搜索树的种数。

示例 1：



> **输入：** n = 3
> **输出：** 5

序列为1-n构成的互不相同的二叉搜索树的个数，依赖于序列1-(n-1)构成的不相同的二叉搜索数的个数...

思路

给定一个有序序列 1⋯n，为了构建出一棵二叉搜索树，我们可以遍历每个数字 i，将该数字作为树根，将 1⋯(i−1) 序列作为左子树，将 (i+1)⋯n 序列作为右子树。接着我们可以按照同样的方式递归构建左子树和右子树。

在上述构建的过程中**，由于根的值不同，因此我们能保证每棵二叉搜索树是唯一的。**

由此可见，原问题可以分解成规模较小的两个子问题，且子问题的解可以复用。因此，我们可以想到使用动态规划来求解本题。

由于序列是有序的，所以我们将 [1,2] 构建不同左子树的数量表示为 G(2), 从 [4,5,6,7] 构建不同右子树的数量表示为 G(4)，注意到 G(n) 和序列的内容无关，只和序列的长度有关。

```python
class Solution:
    def numTrees(self, n: int) -> int:
        dp =[0]*(n+1)
        dp[0] =dp[1] =1 #空树和一个数字构成的二叉搜索树的个数都是1
        for i in range(2,n+1):
            for j in range(1,i+1):  #表示以j为根节点，分别计算左右子树构成二叉搜索树的个数，个数只与序列长度有关，与内容无关
                dp[i] +=dp[j-1]*dp[i-j]  #注意这里是乘法
        return dp[n]
```

首先是背包分类的模板：

1、0/1背包：外循环nums,内循环target,target倒序且target>=nums[i];

2、完全背包：外循环nums,内循环target,target正序且target>=nums[i];

3、考虑顺序的组合背包：外循环target,内循环nums,target正序且target>=nums[i];

4、不考虑顺序的组合背包：外循环nums,内循环target,target正序且target>=nums[i];

5、分组背包：这个比较特殊，需要三重循环：外循环背包bags,内部两层循环根据题目的要求转化为1,2,3三种背包类型的模板

0-1背包问题：

416.分割等和子集

给你一个 **只包含正整数** 的 **非空** 数组 `nums` 。请你判断是否可以将这个数组分割成**两个子集**，使得两个子集的元素和相等。

示例 1：

输入： nums = [1,5,11,5]
输出： true
解释： 数组可以分割成 [1, 5, 5] 和 [11] 。

代码块

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        dp =[False]*10001  #定义
        dp[0] = True  #初始化
        total = sum(nums)
        if total%2!=0:
            return False
        target = total//2
        for num in nums: #先遍历物品
```

```
            for j in range(target,num-1,-1):  #后倒序遍历背包容量
                dp[j] = dp[j] or dp[j-num]
        return dp[target]
```

1049.最后一块石头的重量II

有一堆石头，用整数数组 `stones` 表示。其中 `stones[i]` 表示第 `i` 块石头的重量。

每一回合，从中选出**任意两块石头**，然后将它们一起粉碎。假设石头的重量分别为 `x` 和 `y`，且 `x <= y`。那么粉碎的可能结果如下：

- 如果 `x == y`，那么两块石头都会被完全粉碎；
- 如果 `x != y`，那么重量为 `x` 的石头将会完全粉碎，而重量为 `y` 的石头新重量为 `y-x`。

最后，**最多只会剩下一块** 石头。返回此石头 **最小的可能重量** 。如果没有石头剩下，就返回 `0`。

示例 1：

输入： stones = [2,7,4,1,8,1]
输出： 1
解释：
组合 2 和 4，得到 2，所以数组转化为 [2,7,1,8,1]，
组合 7 和 8，得到 1，所以数组转化为 [2,1,1,1]，
组合 2 和 1，得到 1，所以数组转化为 [1,1,1]，
组合 1 和 1，得到 0，所以数组转化为 [1]，这就是最优值。

代码块

```
class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        #思路：将石头分成两堆，求两堆石头重量差最小值;要让差值小,两堆石头的重量都要接近sum/2;我们假设两堆分别为A,B,A<sum/2,B>sum/2,若A更接近sum/2,B也相应更接近sum/2;进一步转化：将一堆stone放进最大容量为sum/2的背包,求放进去的石头的最大重量MaxWeight(尽可能靠近sum//2的重量),最终答案即为sum-2*MaxWeight.
        target = sum(stones)//2
        dp = [0]*(target+1)
        for s in stones:
            for j in range(target,s-1,-1):
                dp[j] = max(dp[j],dp[j-s]+s)
        return sum(stones)-2*dp[target]
```

494.目标和

给你一个非负整数数组 `nums` 和一个整数 `target` 。

向数组中的每个整数前添加 `'+'` 或 `'-'` ，然后串联起所有整数，可以构造一个 **表达式**：

- 例如，`nums = [2, 1]` ，可以在 `2` 之前添加 `'+'` ，在 `1` 之前添加 `'-'` ，然后串联起来得到表达式 `"+2-1"` 。

返回可以通过上述方法构造的、运算结果等于 `target` 的**不同 表达式 的数目**。

示例 1：

输入： nums = [1,1,1,1,1], target = 3
输出： 5
解释： 一共有 5 种方法让最终目标和为 3 。
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3

代码块

```
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        #思路：相当于nums数组中一部分取正，以哦部分取负；
        #假设：正数部分元素和：p  负数部分(绝对值)元素和：q
        #p+q = sum(nums)  p-q = target
        #p = [sum(nums)+target]//2
        #所以只需要从nums数组中组合(挑选)元素和靠近target的元素即可
        s = sum(nums)+target
        if s<0 or s%2!=0:
            return 0  #表示不能得到p
        target = s//2
        dp =[0]*(target+1)
        dp[0] =1
        for num in nums:
            for j in range(target,num-1,-1):
                dp[j] +=dp[j-num]
        return dp[target]
```

474.一和零

设 $strs[i]$ 中 0 的个数为 $cnt_0[i]$，1 的个数为 $cnt_1[i]$，那么本题相当于：

- 有一个容量为 (m, n) 的背包，至多可以装入 m 个 0 和 n 个 1。现在有 n 个物品，每个物品的体积为 $(cnt_0[i], cnt_1[i])$，表示该物品有 $cnt_0[i]$ 个 0 和 $cnt_1[i]$ 个 1。问：最多可以选多少个物品？

这相当于背包有两种体积（二维），所以在定义状态的时候，相比只有一种体积的 0-1 背包，要多加一个参数。

给你一个二进制字符串数组 `strs` 和两个整数 `m` 和 `n` 。

请你找出并返回 `strs` 的最大子集的长度，该子集中 **最多** 有 `m` 个 `0` 和 `n` 个 `1` 。

如果 `x` 的所有元素也是 `y` 的元素，集合 `x` 是集合 `y` 的 **子集** 。

还是0-1背包问题，只不过换成了多个背包容量，多加一层for循环

代码块

```
输入: strs = ["10", "0001", "111001", "1", "0"], m = 5, n = 3
输出: 4
解释: 最多有 5 个 0 和 3 个 1 的最大子集是 {"10","0001","1","0"} ，因此答案是 4 。
其他满足题意但较小的子集包括 {"0001","1"} 和 {"10","1","0"} 。{"111001"} 不满足题意，
因为它含 4 个 1 ，大于 n 的值 3 。
```

代码块

```python
class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:
        #0-1背包问题
        dp=[[0]*(n+1) for _ in range(m+1)]
        for s in strs:
            cnt0 = s.count('0')
            cnt1 = len(s) -cnt0
            for j in range(m,cnt0-1,-1):  #倒叙
                for k in range(n,cnt1-1,-1):
                    dp[j][k] =max(dp[j][k],dp[j-cnt0][k-cnt1]+1)
        return dp[m][n]
```

完全背包问题：

322.零钱兑换

给你一个整数数组 `coins` ，表示不同面额的硬币；以及一个整数 `amount` ，表示总金额。

计算并返回可以凑成总金额所需的 **最少的硬币个数** 。如果没有任何一种硬币组合能组成总金额，返回 `-1` 。

你可以认为每种硬币的数量是无限的。

示例 1：

代码块

```
输入: coins = [1, 2, 5], amount = 11输出: 3
```

```
2   解释: 11 = 5 + 5 + 1
```

代码块

```
1   class Solution:
2       def coinChange(self, coins: List[int], amount: int) -> int:
3           #完全背包问题：每个物品可以取无限次
4           dp = [float('inf')]*(amount+1) #表示凑成i金额所需的最少的硬币个数
5           dp[0] = 0
6           for coin in coins:
7               for j in range(coin,amount+1):  #确保j>coin
8                   dp[j] = min(dp[j],dp[j-coin]+1)
9           return dp[amount] if dp[amount]!=float('inf') else -1
```

279.完全平方数

给你一个整数 `n` ，返回 *和为* `n` *的完全平方数的**最少数量***。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如， `1` 、 `4` 、 `9` 和 `16` 都是完全平方数，而 `3` 和 `11` 不是。

示例 1：

代码块

```
1   输入: n = 12输出: 3
2   解释: 12 = 4 + 4 + 4
```

代码块

```
1   class Solution:
2       def numSquares(self, n: int) -> int:
3           #完全背包问题
4           #问题转换物品：[1,isqrt(n)],每次取物品的平方当作物品的重量，背包容量为n
5           dp =[float('inf')]*(n+1)
6           dp[0] =0
7           dp[1] =1
8           for num in range(1,isqrt(n)+1):
9               for j in range(num**2,n+1):  #完全背包问题，正序遍历
10                  dp[j]=min(dp[j],dp[j-num**2]+1)
11          return dp[n]
```

377.组合总结IV

给你一个由 **不同** 整数组成的数组 `nums` ，和一个目标整数 `target` 。请你从 `nums` 中找出并返回总和为 `target` 的元素排列的个数。

题目数据保证答案符合 32 位整数范围。

代码块

```
输入：nums = [1,2,3], target = 4
输出：7
解释：
所有可能的组合为：
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
(2, 1, 1)
(2, 2)
(3, 1)
请注意，顺序不同的序列被视作不同的组合。
```

代码块

```
class Solution:
    #完全背包问题：考虑到顺序组合问题，外层target,内层nums正序
    def combinationSum4(self, nums: List[int], target: int) -> int:
        dp =[0]*(target+1)  #dp[i]:从 nums 中找出并返回总和为i的元素排列的个数。
        dp[0] =1
        #动态规划的边界是 dp[0]=1。只有当不选取任何元素时，元素之和才为 0，因此只有 1 种方案。
        for j in range(1,target+1):
            for num in nums:
                if num<=j:
                    dp[j] +=dp[j-num]
        return dp[target]
```

518.零钱兑换II

给你一个整数数组 `coins` 表示不同面额的硬币，另给一个整数 `amount` 表示总金额。

请你计算并返回可以**凑成总金额的硬币组合数**。如果任何硬币组合都无法凑出总金额，返回 `0` 。

假设每一种面额的硬币有无限个。

题目数据保证结果符合 32 位带符号整数.

代码块

```
输入：amount = 5, coins = [1, 2, 5]
```

```
输出：4
解释：有四种方式可以凑成总金额：
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1
```

代码块

```python
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        #不考虑顺序问题的组合——完全背包：外循环coins,内循环target正序
        dp=[0]*(amount+1)
        dp[0] = 1  #不选任何一种硬币也是一种方案
        for coin in coins:
            for j in range(coin,amount+1):
                dp[j]+=dp[j-coin]
        return dp[amount]
```

分组背包，有n个组，每个组只能选一次，每个组有多个物品(k)，问填满容量为target的背包有几种方式。解题思路是：

a. 01背包的解法里套一个内循环，用于循环组内的物品

b. 同时有个细节，传统的01背包可以装到背包里，也可以不装到背包里，而这套题不行，必须要装，所以上一次的状态不能累加到当前的状态里，内循环之前要清零上一次的状态。换句话说，如果不清零，就意味着当前组/筛子可以跳过

139.单词拆分

给你一个字符串 `s` 和一个字符串列表 `wordDict` 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 `s` 则返回 `true`。

注意： 不要求字典中出现的单词全部都使用，并且字典中的单词**可以重复使用**。

示例 1：

输入: s = "leetcode", wordDict = ["leet", "code"]
输出: true
解释: 返回 true 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

代码块

```python
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        #针对s序列不断枚举开始和结束索引的位置
```

```
        #dp[i]:s[0:i+1]该索引是否可以由wordDict中的元素构成
        n = len(s)
        dp =[False]*(n+1)
        dp[0] = True
        for i in range(n):  #遍历开始索引 i
            for j in range(i+1,n+1): #遍历结束索引 j
                if dp[i] and (s[i:j] in wordDict):
                    dp[j] = True
        return dp[n]
```

1155.掷骰子等于目标和的方法数

这里有 `n` 个一样的骰子，每个骰子上都有 `k` 个面，分别标号为 `1` 到 `k` 。

给定三个整数 `n` 、 `k` 和 `target` ，请返回投掷骰子的所有可能得到的结果（共有 `k(n)` 种方式），使得骰子面朝上的数字总和等于 `target` 。

由于答案可能很大，你需要对 `10(9) + 7` **取模**。

代码块

```
1  输入：n = 1, k = 6, target = 3
2  输出：1
3  解释：你掷了一个有 6 个面的骰子。
4  得到总和为 3 的结果的方式只有一种。
```

代码块

```
1
```

打家劫舍：

198.打家劫舍

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警**。

给定一个代表每个房屋存放金额的非负整数数组，计算你 **不触动警报装置的情况下** ，一夜之内能够偷窃到的**最高金额。**

示例 1：

输入： [1,2,3,1]
输出： 4

解释： 偷窃 1 号房屋 (金额 = 1) ，然后偷窃 3 号房屋 (金额 = 3)。
偷窃到的最高金额 = 1 + 3 = 4 。

代码块

```
 #动态规划:
    #找到子问题：对于n间房子，如果最后一间不偷，则最大偷窃金额为偷前n-1间房子的最大金额;
    #            如果偷最后一间，第n-1间房间不能偷，则最大偷窃金额为偷前n-2间房子的最大金额。
  class Solution:
     def rob(self, nums: List[int]) -> int:
         n = len(nums)
         dp =[0]*(n+1)  #dp[i]:偷前i间可以获得的最大金额
         dp[0] =0
         dp[1] =nums[0]
         for i in range(2,n+1):
             dp[i] = max(dp[i-1],dp[i-2]+nums[i-1])  #这里nums[i-1]:表示第i间房子的金额
         return dp[n]
```

213.打家劫舍II

你是一个专业的小偷，计划偷窃沿街的房屋，每间房内都藏有一定的现金。这个地方所有的房屋都 **围成一圈** ，这意味着第一个房屋和最后一个房屋是紧挨着的。同时，相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警** 。

给定一个代表每个房屋存放金额的非负整数数组，计算你 **在不触动警报装置的情况下** ，今晚能够偷窃到的最高金额。

示例 1：

输入： nums = [2,3,2]
输出： 3
解释： 你不能先偷窃 1 号房屋（金额 = 2），然后偷窃 3 号房屋（金额 = 2）, 因为他们是相邻的。

代码块

```
class Solution:
    #表示:如果偷nums[0],则nums[1],nums[n-1]不能偷。问题转换成偷nums[2]-nums[n-2]房间的最大金额
    #如何不偷nums[0],则nums[1],nums[n-1]都能偷。问题转换成偷nums[1]-nums[n-1]房间的最大金额
    #由此环形问题转化成了非环形问题
    def rob1(self, nums: List[int]) -> int:  #这个函数用来求最大金额
        n = len(nums)
        if n==0:   #要判断数组是否为空，为空的话不存在dp[1]=nums[0]
            return 0
```

```
        dp =[0]*(n+1)
        dp[0]=0 #表示一间房间不偷
        dp[1]=nums[0]
        for i in range(2,n+1):
            dp[i]=max(dp[i-1],nums[i-1]+dp[i-2])
        return dp[n]
    def rob(self,nums:List[int])->int:
        return max(nums[0]+self.rob1(nums[2:-1]),self.rob1(nums[1:]))
```

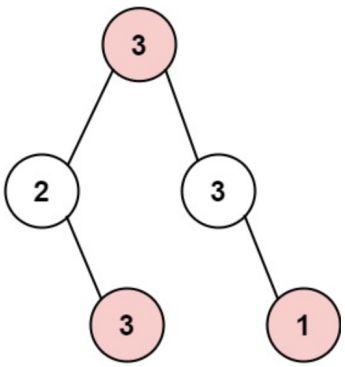
337.打家劫舍III

小偷又发现了一个新的可行窃的地区。这个地区只有一个入口，我们称之为 `root` 。

除了 `root` 之外，每栋房子有且只有一个“父“房子与之相连。一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。 如果 **两个直接相连的房子在同一天晚上被打劫** ，房屋将自动报警。

给定二叉树的 `root` 。返回 ***在不触动警报的情况下***，*小偷能够盗取的最高金额*。

示例 1:



> **输入：** root = [3,2,3,null,3,null,1]
> **输出：** 7
> **解释：** 小偷一晚能够盗取的最高金额 3 + 3 + 1 = 7

代码块

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rob(self, root: Optional[TreeNode]) -> int:
        def dfs(node:Optional[TreeNode])->int:
            if node is None:
                return 0,0
            l_rob,l_not_rob = dfs(node.left)
            r_rob,r_not_rob =dfs(node.right)
            rob = node.val+l_not_rob+r_not_rob
```

```
            not_rob = max(l_not_rob,l_rob)+max(r_not_rob,r_rob)
            return rob,not_rob
        return max(dfs(root))  #根节点偷与不偷的最大值
```

股票问题：

121.买股票的最佳时机

给定一个数组 `prices` ，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。

你只能选择 **某一天** 买入这只股票，并选择在 **未来的某一个不同的日子** 卖出该股票。设计一个算法来计算你所能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 `0` 。

示例 1：

输入： [7,1,5,3,6,4]
输出： 5
解释： 在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 。
　注意利润不能是 7-1 = 6, 因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

代码块

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        #价格最低的时候买
        #价格最高的时候卖
        min_price =inf
        max_profit =0
        for price in prices:
            min_price = min(min_price,price)
            max_profit = max(max_profit,price-min_price)
        return max_profit
```

122.买股票的最佳时机II

给你一个整数数组 `prices` ，其中 `prices[i]` 表示某支股票第 `i` 天的价格。

在每一天，你可以决定是否购买和/或出售股票。你在任何时候 **最多** 只能持有 **一股** 股票。然而，你可以在 **同一天** **多次买卖该股票**，但要确保你持有的股票不超过一股。

返回 *你能获得的 **最大** 利润* 。

示例 1：

输入： prices = [7,1,5,3,6,4]

输出： 7
解释： 在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5 - 1 = 4。
随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出, 这笔交易所能获得利润 = 6 - 3 = 3。
最大总利润为 4 + 3 = 7 。

代码块

```
class Solution:
 #动态规划: dp[i][0]第i天不持有股票获得的最大利润;dp[i][1]第i天持有股票获得的最大利润
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        dp = [[0]*2 for _ in range(n)]
        dp[0][0] =0 #第0天不持有股票
        dp[0][1] =-prices[0] #第0天持有股票
        for i in range(1,n):
            dp[i][0] = max(dp[i-1][0],dp[i-1][1]+prices[i]) #表示第i天不持有股票所能获得的最大利润
            dp[i][1] = max(dp[i-1][1],dp[i-1][0]-prices[i])
        return dp[n-1][0]
```

123.买股票的最佳时机III

给定一个数组，它的第 `i` 个元素是一支给定的股票在第 `i` 天的价格。

设计一个算法来计算你所能获取的最大利润。你**最多可以完成 两笔 交易。**

注意： 你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入： prices = [3,3,5,0,0,3,1,4]
输出： 6
解释： 在第 4 天（股票价格 = 0）的时候买入，在第 6 天（股票价格 = 3）的时候卖出，这笔交易所能获得利润 = 3-0 = 3 。
　随后，在第 7 天（股票价格 = 1）的时候买入，在第 8 天 （股票价格 = 4）的时候卖出，这笔交易所能获得利润 = 4-1 = 3 。

可以进行几次交易，就设置几种状态

代码块

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
```

```
        #dp[i][0]:第一次买入的最大收益
        #dp[i][1]:第一次卖出的最大收益
        #dp[i][2]:第二次买入的最大收益
        #dp[i][3]:第二次卖出的最大收益
        n = len(prices)
        dp = [[0]*4 for _ in range(n)]
        dp[0][0] = -prices[0]
        dp[0][1] = 0
        dp[0][2] = -prices[0]
        dp[0][3] = 0
        for i in range(1,n):
            dp[i][0] = max(dp[i-1][0],-prices[i])  #第一次买入
            dp[i][1] = max(dp[i-1][1],dp[i-1][0]+prices[i])
            dp[i][2] = max(dp[i-1][2],dp[i][1]-prices[i])  #在第一次卖出的基础上进行二次买入
            dp[i][3] = max(dp[i-1][3],dp[i-1][2]+prices[i]) #在第二次买入的基础上进行卖出
        return dp[n-1][3]

```

188.买股票的最佳时机IV

给你一个整数数组 `prices` 和一个整数 `k` ，其中 `prices[i]` 是某支给定的股票在第 `i` 天的价格。

设计一个算法来计算你所能获取的最大利润。你最多可以完成 `k` 笔交易。也就是说，你最多可以买 `k` 次，卖 `k` 次。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1：

输入：k = 2, prices = [2,4,1]
输出：2
解释：在第 1 天 (股票价格 = 2) 的时候买入，在第 2 天 (股票价格 = 4) 的时候卖出，这笔交易所能获得利润 = 4-2 = 2 。

代码块

```
class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        # dp[i][j][0]: 第 i 天，完成 j 次交易，手里没股票
        # dp[i][j][1]: 第 i 天，完成 j 次交易，手里有股票
        n = len(prices)
        dp=[[[0]*2 for _ in range(k+1)] for _ in range(n)]
        #最多可以完成 k 笔交易，也可以不进行交易k=0
        for j in range(k+1):
            dp[0][j][0] =0  #第0天的j次交易卖出的收益
```

```
            dp[0][j][1] =-prices[0] #第0天的j次交易买入的收益
        for i in range(1,n):
            for j in range(1,k+1):
                #不操作/操作
                dp[i][j][1] = max(dp[i-1][j][1],dp[i-1][j-1][0]-prices[i])  #有股
                #只有在上笔交易不持有股票时，才能买入
                dp[i][j][0] = max(dp[i-1][j][0],dp[i-1][j][1]+prices[i])#无股

        return max(dp[n - 1][j][0] for j in range(k + 1))
```

309.买卖股票的最佳时机含冷冻期

给定一个整数数组 `prices` ，其中第 `prices[i]` 表示第 `i` 天的股票价格 。

设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：

- 卖出股票后，你无法在第二天买入股票 (即冷冻期为 1 天)。

注意： 你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入: prices = [1,2,3,0,2]
输出: 3
解释: 对应的交易状态为: [买入, 卖出, 冷冻期, 买入, 卖出]

代码块

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        #在卖出股票时会进入冷冻期
        n =len(prices)
        if n == 1:
            return 0
        dp =[[0]*2 for _ in range(n)]
        #dp[i][0]:第i天不持有股票
        #dp[i][1]: 第i天持有股票
        dp[0][0] =0
        dp[0][1] = -prices[0]
        dp[1][0] = max(0, prices[1] - prices[0])#不持有
        dp[1][1] =max(-prices[0], -prices[1])#持有
        for i in range(2,n):
            dp[i][0] = max(dp[i-1][0],dp[i-1][1]+prices[i])
            dp[i][1] = max(dp[i-1][1],dp[i-2][0]-prices[i])
        return dp[n-1][0]
```

714.买卖股票的最佳实际含手续费

给定一个整数数组 `prices` ，其中 `prices[i]` 表示第 `i` 天的股票价格；整数 `fee` 代表了交易股票的手续费用。

你可以**无限次地完成交易**，但是你每笔交易都需要付手续费。如果你已经购买了一个股票，在卖出它之前你就不能再继续购买股票了。

返回获得利润的最大值。

注意：这里的一笔交易指买入持有并卖出股票的整个过程，每笔交易你只需要为支付一次手续费。

示例 1：

输入： prices = [1, 3, 2, 8, 4, 9], fee = 2
输出： 8
解释： 能够达到的最大利润:
在此处买入 prices[0] = 1
在此处卖出 prices[3] = 8
在此处买入 prices[4] = 4
在此处卖出 prices[5] = 9
总利润: ((8 - 1) - 2) + ((9 - 4) - 2) = 8

代码块

```
class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        #每笔交易你只需要为支付一次手续费：在卖出的时候要付手续费
        n =len(prices)
        dp=[[0]*2 for _ in range(n)]
        dp[0][0] =0   #不持有
        dp[0][1] =-prices[0]  #持有
        for i in range(1,n):
            dp[i][0] = max(dp[i-1][0],dp[i-1][1]+prices[i]-fee)
            dp[i][1] = max(dp[i-1][1],dp[i-1][0]-prices[i])
        return dp[n-1][0]
```

子序列问题：

1.最长递增子序列

给你一个整数数组 `nums` ，找到其中最长严格递增子序列的长度。

子序列 是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，`[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

代码块

```
输入：nums = [10,9,2,5,3,7,101,18]
输出：4
解释：最长递增子序列是 [2,3,7,101]，因此长度为 4 。
```

代码块

```python
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        #dp[i]: 表示以nums[i]结尾的最长子序列长度
        n =len(nums)
        dp =[1]*n
        for i in range(n):
            for j in range(i):
                if nums[j]<nums[i]:
                    dp[i] = max(dp[i],dp[j]+1)
        return max(dp)
```

674.最长连续递增序列

给定一个**未经排序**的整数数组，找到最长且 **连续递增的子序列**，并返回该序列的长度。

连续递增的子序列 可以由两个下标 `l` 和 `r` （`l < r`）确定，如果对于每个 `l <= i < r`，都有 `nums[i] < nums[i + 1]` ，那么子序列 `[nums[l], nums[l + 1], ..., nums[r - 1], nums[r]]` 就是连续递增子序列。

示例 1：

输入： nums = [1,3,5,4,7]
输出： 3
解释： 最长连续递增序列是 [1,3,5], 长度为3。
尽管 [1,3,5,7] 也是升序的子序列, 但它不是连续的，因为 5 和 7 在原数组里被 4 隔开。

代码块

```python
#1.动态规划的思想
class Solution:
    def findLengthOfLCIS(self, nums: List[int]) -> int:
        n =len(nums)
        dp =[1]*n
        for i in range(1,n):
            if nums[i-1]<nums[i]:  #保证了连续性
                dp[i] = max(dp[i],dp[i-1]+1)
        return max(dp)
#2.使用双指针的方法来做
class Solution:
    def findLengthOfLCIS(self, nums: List[int]) -> int:
```

```
        n =len(nums)
        ans =1  #最终的最长序列长度
        count =1 #记录连续递增序列的长度，一旦不连续就设置为1，表示重新开始一个新的连续序列
        for i in range(n-1):
            if nums[i+1]>nums[i]:
                count +=1
            else:
                count =1
            ans = max(ans,count)
        return ans
```

718.最长重复子数组

给两个整数数组 `nums1` 和 `nums2` ，返回 *两个数组中* ***公共的***、*长度最长的子数组的长度*。

示例 1：

输入： nums1 = [1,2,3,2,1], nums2 = [3,2,1,4,7]
输出： 3
解释： 长度最长的公共子数组是 [3,2,1]

代码块

```
class Solution:
    def findLength(self, nums1: List[int], nums2: List[int]) -> int:
        #子数组在原数组中连续
        #f[i][j]:以 nums1[i-1] 和 nums2[j-1] 结尾的最长公共连续子数组长度
        ans =0
        n,m = len(nums1), len(nums2)
        f = [[0] * (m + 1) for _ in range(n + 1)]
        for i, x in enumerate(nums1):
            for j, y in enumerate(nums2):
                if x == y:
                    f[i + 1][j + 1] = f[i][j] + 1
                    ans = max(ans,f[i+1][j+1])
        return ans
```

1143.最长公共子序列

给定两个字符串 `text1` 和 `text2` ，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列** ，返回 `0` 。

一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

- 例如， `"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。

两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

输入： text1 = "abcde", text2 = "ace"
输出： 3
解释： 最长公共子序列是 "ace" ，它的长度为 3 。

$$dp[i][j] = \begin{cases} dp[i-1][j-1]+1, & text_1[i-1] = text_2[j-1] \\ \max(dp[i-1][j], dp[i][j-1]), & text_1[i-1] \neq text_2[j-1] \end{cases}$$

代码块

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        #子序列：不要求连续
        #dp[i][j]:text1[:i] 和 text2[:j] 的最长公共子序列长度
        m,n = len(text1),len(text2)
        dp =[[0]*(n+1) for _ in range(m+1)]
        for i in range(1,m+1):
            for j in range(1,n+1):
                if text1[i-1]==text2[j-1]:
                    dp[i][j] = dp[i-1][j-1]+1
                else:
                    dp[i][j] = max(dp[i-1][j],dp[i][j-1])
        return dp[m][n]

```

1035.不相交的线

在两条独立的水平线上按给定的顺序写下 `nums1` 和 `nums2` 中的整数。

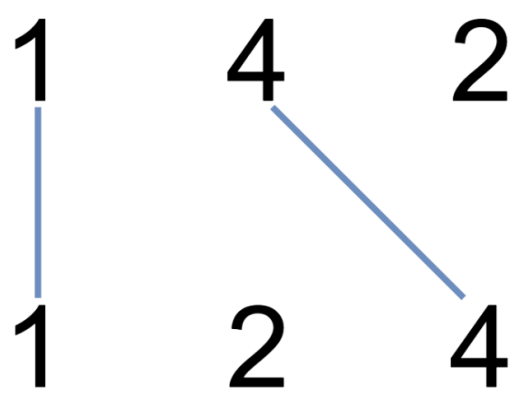
现在，可以绘制一些连接两个数字 `nums1[i]` 和 `nums2[j]` 的直线，这些直线需要同时满足：

- `nums1[i] == nums2[j]`
- 且绘制的直线不与任何其他连线（非水平线）相交。

请注意，连线即使在端点也不能相交：每个数字只能属于一条连线。

以这种方法绘制线条，并返回可以绘制的最大连线数。

示例 1：



> **输入：** nums1 = [1,4,2], nums2 = [1,2,4]
> **输出：** 2
> **解释：** 可以画出两条不交叉的线，如上图所示。
> 但无法画出第三条不相交的直线，因为从 nums1[1]=4 到 nums2[2]=4 的直线将与从 nums1[2]=2 到 nums2[1]=2 的直线相交。

代码块

```
class Solution:
    def maxUncrossedLines(self, nums1: List[int], nums2: List[int]) -> int:
        #这道题转化一下:实际就是求两个数组的公共子序列（保证相对位置）
        m,n =len(nums1),len(nums2)
        dp =[[0]*(n+1) for _ in range(m+1)]
        ans =0
        for i in range(1,m+1):
            for j in range(1,n+1):
                if nums1[i-1]==nums2[j-1]:
                    dp[i][j] = dp[i-1][j-1]+1
                else:
                    dp[i][j] = max(dp[i-1][j],dp[i][j-1])
        return dp[m][n]
```

53.最大子序和

给你一个整数数组 `nums` ，请你找出一个具有最大和的**连续子数组**（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

示例 1：

输入： nums = [-2,1,-3,4,-1,2,1,-5,4]
输出： 6
解释： 连续子数组 [4,-1,2,1] 的和最大，为 6 。

dp[i]:以nums[i]结尾的连续子数组的最大和

如果dp[i-1]<=0 :dp[i]=nums[i]

如果dp[i-1]>0:dp[i]=dp[i-1]+nums[i]

代码块

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        n =len(nums)
        dp =[0]*n  #dp[i]:以nums[i]结尾的子数组的最大和
        dp[0] = nums[0]
        for i in range(1,n):
            if dp[i-1]>=0:
                dp[i] = dp[i-1]+nums[i]
            else:
                dp[i] = nums[i]
        return max(dp)
```

392.判断子序列

给定字符串 **s** 和 **t** ，判断 **s** 是否为 **t** 的子序列。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，`"ace"` 是 `"abcde"` 的一个子序列，而 `"aec"` 不是）。

示例 1：

输入： s = "abc", t = "ahbgdc"
输出： true

代码块

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        #双指针的做法
        #拿着s作为标准答案，去找t中的序列
        if not s:
            return True
        i=0
        for c in t:
            if c==s[i]:
                i+=1
            if i==len(s):
                return True
        return False
```

583.两个字符串的删除操作

给定两个单词 `word1` 和 `word2` ，返回使得 `word1` 和 `word2` **相同**所需的**最小步数**。

每步 可以删除任意一个字符串中的一个字符。

示例 1:

输入: word1 = "sea", word2 = "eat"
输出: 2
解释: 第一步将 "sea" 变为 "ea" ，第二步将 "eat "变为 "ea"

给定两个字符串 word1和 word2，分别删除若干字符之后使得两个字符串相同，则剩下的字符为两个字符串的公共子序列。为了使删除操作的次数最少，剩下的字符应尽可能多。当剩下的字符为两个字符串的最长公共子序列时，删除操作的次数最少。因此，可以计算两个字符串的最长公共子序列的长度，然后分别计算两个字符串的长度和最长公共子序列的长度之差，即为两个字符串分别需要删除的字符数，两个字符串各自需要删除的字符数之和即为最少的删除操作的总次数。

代码块

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        #问题转化: 求最长公共子序列
        #dp[i][j]:表示word1[:i]和word2[:j]序列的最长公共序列长度
        m,n =len(word1),len(word2)
        dp=[[0]*(n+1) for _ in range(m+1)]
        for i in range(1,m+1):
            for j in range(1,n+1):
                if word1[i-1]==word2[j-1]:
                    dp[i][j] =dp[i-1][j-1]+1
                else:
                    dp[i][j]=max(dp[i][j-1],dp[i-1][j])
        lcs = dp[m][n]
        return m+n-2*lcs
```

72.编辑距离

给你两个单词 `word1` 和 `word2`， *请返回将* `word1` *转换成* `word2` *所使用的**最少操作数*** 。

你可以对一个单词进行如下三种操作:

- 插入一个字符
- 删除一个字符
- 替换一个字符

示例 1：

输入： word1 = "horse", word2 = "ros"
输出： 3
解释：
horse -> rorse (将 'h' 替换为 'r')
rorse -> rose (删除 'r')
rose -> ros (删除 'e')

代码块

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        # Ikaruga的解析,动态规划问题
        #1.如果word1或者word2为空
        #dp[i][j] 代表 word1 中前 i 个字符，变换到 word2 中前 j 个字符，最短需要操作的次数
        n = len(word1)
        m = len(word2)
        dp =[[0]*(m+1) for _ in range(n+1)]
        for i in range(n+1):
            dp[i][0] = i
        for j in range(m+1):
            dp[0][j] = j
        for i in range(1,n+1):
            for j in range(1,m+1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    # 三种操作取最小值
                    # 1. 替换：dp[i-1][j-1] + 1
                    # 2. 删除：dp[i-1][j] + 1
                    # 3. 插入：dp[i][j-1] + 1
                    dp[i][j] = min(
                        dp[i - 1][j - 1],  # 替换，保证word1[:i-1]和word2[:j-1]相等，这样替换word1[i-1],才会有word1[:i]==word2[:j]
                        dp[i - 1][j],      # 删除word1多出的元素之前，要保证前面的序列和word2一样
                        dp[i][j - 1]       # 插入
                    ) + 1
        return dp[n][m]
```

5.最长回文子串

给你一个字符串 `s`，找到 `s` 中最长的 回文 子串。

示例 1：

输入： s = "babad"
输出： "bab"
解释： "aba" 同样是符合题意的答案。

代码块

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n =len(s)
        #一个字符为中心
        ans_left=0
        ans_right=0
        for i in range(n):
            l=r =i
            while l>=0 and r<n and s[l]==s[r]:
                l-=1
                r+=1
            if r-l-1>ans_right-ans_left:
                ans_left,ans_right = l+1,r
        #两个字符为中心
        for j in range(n-1):
            l,r = j ,j+1
            while l>=0 and r<n and s[l]==s[r]:
                l-=1
                r+=1
            if r-l-1>ans_right-ans_left:
                ans_left,ans_right = l+1,r
        return s[ans_left:ans_right]

#动态规划
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        ans_left ,ans_right =0,0
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                if s[i] == s[j]:    #如果s[i]!=s[j]:则dp[i][j] =False
                    if j-i<=1:    #一个或者两个字符的情况
                        dp[i][j] = True
                    elif j-i>1:         #大于两个字符的情况,要保证中间的子串是回文子串
                        dp[i][j] = dp[i + 1][j - 1]
                if dp[i][j] and j-i+1 >ans_right-ans_left+1:
                    ans_left,ans_right = i,j
        return s[ans_left:ans_right+1]
```

647.回文子串

给你一个字符串 `s` ，请你统计并返回这个字符串中 **回文子串** 的数目。

回文字符串 是正着读和倒过来读一样的字符串。

子字符串 是字符串中的由**连续字符**组成的一个序列。

示例 1：

输入： s = "abc"
输出： 3
解释： 三个回文子串: "a", "b", "c"

和最长回文子串题目类似，都遍历了每个子串 ，只不过这里只需统计回文串的个数

代码块

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        n =len(s)
        ans =0
        #一个字符为中心
        for i in range(n):
            l=r =i
            while l>=0 and r<n and s[l]==s[r]:
                l-=1
                r+=1
                ans +=1
        #两个字符为中心
        for j in range(n-1):
            l,r = j ,j+1
            while l>=0 and r<n and s[l]==s[r]:
                l-=1
                r+=1
                ans +=1
        return ans

#动态规划的思想来做:
class Solution:
    def countSubstrings(self, s: str) -> int:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        ans = 0
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                if s[i] == s[j]:
```

```
                    if j-i<=1:    #一个或者两个字符的情况
                        dp[i][j] = True
                    elif j-i>1:         #大于两个字符的情况,要保证中间的子串是回文子串
                        dp[i][j] = dp[i + 1][j - 1]
                if dp[i][j]:
                    ans += 1

        return ans

```

516.最长回文子序列

给你一个字符串 `s` ，找出其中最长的回文子序列，并返回该序列的长度。

子序列定义为：不改变剩余字符顺序的情况下，删除某些字符或者不删除任何字符形成的一个序列。

示例 1：

输入： s = "bbbab"
输出： 4
解释： 一个可能的最长回文子序列为 "bbbb" 。

代码块

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        #dp[i][j]:字符串 s[i:j+1] 这个区间内，最长回文子序列的长度
        n = len(s)
        dp = [[0]*n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1
        #i,j是子序列区间的两个端点
        for i in range(n-1,-1,-1):  #由于dp[i][j] = dp[i+1][j-1]+2 ,所以i要从大到小计算，这样子问题才已经算过。
            for j in range(i+1,n):
                if s[i]==s[j]:
                    dp[i][j] = dp[i+1][j-1]+2
                else:
                    dp[i][j] =max(dp[i][j-1],dp[i+1][j])
        return dp[0][n-1]

```